9.3 INSERTION SORT

Suppose an array A with n elements A[1], A[2], ..., A[N] is in memory. The insertion sort algorithm scans A from A[1] to A[N], inserting each element A[K] into its proper position in the previously sorted subarray A[1], A[2], ..., A[K – 1]. That is:

Pass 1. A[1] by itself is trivially sorted.

Pass 2. A[2] is inserted either before or after A[1] so that: A[1], A[2] is sorted.

Pass 3. A[3] is inserted into its proper place in A[1], A[2], that is, before A[1], between A[1] and A[2], or after A[2], so that: A[1], A[2], A[3] is sorted.

Pass 4. A[4] is inserted into its proper place in A[1], A[2], A[3] so that:

A[1], A[2], A[3], A[4] is sorted.

.....

Pass N. A[N] is inserted into its proper place in A[1], A[2], ..., A[N – 1] so that:

A[1], A[2], ..., A[N] is sorted.

This sorting algorithm is frequently used when n is small. For example, this algorithm is very popular with bridge players when they are first sorting their cards.

There remains only the problem of deciding how to insert A[K] in its proper place in the sorted subarray A[1], A[2], ..., A[K – 1]. This can be accomplished by comparing A[K] with A[K – 1], comparing A[K] with A[K – 2], comparing A[K] with A[K – 3], and so on, until first meeting an

element A[J] such that A[J] ≤ A[K]. Then each of the elements A[K – 1], A[K – 2], ..., A[J + 1] is moved forward one location, and A[K] is then inserted in the J + 1st position in the array.

The algorithm is simplified if there always is an element A[J] such that A[J] ≤ A[K]; otherwise we must constantly check to see if we are comparing A[K] with A[1]. This condition can be accomplished by introducing a sentinel element A[0] = –∞ (or a very small number).

Example 9.4

Suppose an array A contains 8 elements as follows:

77, 33, 44, 11, 88, 22, 66, 55

Figure 9.3 illustrates the insertion sort algorithm. The circled element indicates the A[K] in each pass of the algorithm, and the arrow indicates the proper place for inserting A[K].

Pass	A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	A[6]	A[7]	A[8]
K = 1:	–∞	(77)	33	44	11	88	22	66	55
K = 2:	–∞	77	(33)	44	11	88	22	66	55
K = 3:	–∞	33	77	(44)	11	88	22	66	55
K = 4:	–∞	33	44	77	(11)	88	22	66	55
K = 5:	–∞	11	33	44	77	(88)	22	66	55
K = 6:	–∞	11	33	44	77	88	(22)	66	55
K = 7:	–∞	11	22	33	44	77	88	(66)	55
K = 8:	–∞	11	22	33	44	66	77	88	(55)
Sorted:	–∞	11	22	33	44	55	66	77	88

Fig. 9.3 *Insertion Sort for n = 8 Items*

The formal statement of our insertion sort algorithm follows.

Algorithm 9.1: (Insertion Sort) INSERTION(A, N).
This algorithm sorts the array A with N elements.

1. Set A[0] := –∞. [Initializes sentinel element.]
2. Repeat Steps 3 to 5 for K = 2, 3, ..., N:
3. Set TEMP := A[K] and PTR := K – 1.
4. Repeat while TEMP < A[PTR]:
   (a) Set A[PTR + 1] := A[PTR]. [Moves element forward.]
   (b) Set PTR := PTR – 1.
   [End of loop.]
5. Set A[PTR + 1] := TEMP. [Inserts element in proper place.]
   [End of Step 2 loop.]
6. Return.

Observe that there is an inner loop which is essentially controlled by the variable PTR, and there is an outer loop which uses K as an index.

Complexity of Insertion Sort

The number $f(n)$ of comparisons in the insertion sort algorithm can be easily computed. First of all, the worst case occurs when the array A is in reverse order and the inner loop must use the maximum number K – 1 of comparisons. Hence

$$f(n) = 1 + 2 + \cdots + (n-1) = \frac{n(n-1)}{2} = O(n^2)$$

Furthermore, one can show that, on the average, there will be approximately (K – 1)/2 comparisons in the inner loop. Accordingly, for the average case,

$$f(n) = \frac{1}{2} + \frac{2}{2} + \cdots + \frac{n-1}{2} = \frac{n(n-1)}{4} = O(n^2)$$

Thus the insertion sort algorithm is a very slow algorithm when n is very large.

The above results are summarized in the following table:

Algorithm	*Worst Case*	*Average Case*
Insertion Sort	$\frac{n(n-1)}{2} = O(n^2)$	$\frac{n(n-1)}{4} = O(n^2)$

Remark: Time may be saved by performing a binary search, rather than a linear search, to find the location in which to insert A[K] in the subarray A[1], A[2], …, A[K – 1]. This requires, on the average, log K comparisons rather than (K – 1)/2 comparisons. However, one still needs to move (K – 1)/2 elements forward. Thus the order of complexity is not changed. Furthermore, insertion sort is usually used only when n is small, and in such a case, the linear search is about as efficient as the binary search.

9.4 SELECTION SORT

Suppose an array A with n elem…